React

```
import React, { useEffect, useState } from "react";


const container = {
    backgroundColor:"yellow",
    margin:"10px"
}

const Usehooooks = () => {
  const [count, setCount] = useState([]);

  useEffect(() => {
    fetch("https://jsonplaceholder.typicode.com/posts")
      .then((res) => res.json())
      .then((data) => setCount(data));
  }, []);

  return (
    <div>
      <h1>Use Effect</h1>
      {/* <h1>{count}</h1> */}

      {
        count.map((ele)=>{
            return <div key={ele.id}  style={container}>
                <h2>{ele.title}</h2>
            </div>
        })
      }
      <button onClick={() =>  setCount(count + 1)}>Increment</button>
    </div>
  );
};

export default Usehooooks;
```

```
import React, { useReducer } from "react";

const UseReducerrr = () => {
  const reducer = (state, action) => {
    if (action.type == "increment") {
      console.log(action.payload);
```

```
      return { count: state.count + 1 };
    } else if (action.type == "decrement") {
      return { count: state.count - 1 };
    } else if (action.type == "reset") {
      return { count: 0 };
    }
  };
  const [state, dispatch] = useReducer(reducer, { count: 0 });

  return (
    <div>
      <h2>Use reducer</h2>

      <p>{state.count}</p>

      <button onClick={() => dispatch({ type: "increment", payload: 20 })}>
        {" "}
        Increment
      </button>

      <button onClick={() => dispatch({ type: "decrement"
})}>Decrement</button>
      <button onClick={() => dispatch({ type: "reset" })}>Reset</button>
    </div>
  );
};

export default UseReducerrr;
```

```
import React, { useRef } from "react";

const Userefff = () => {
  const inputref = useRef();

  const focusInput = () => {
    inputref.current.focus();
  };

  return (
    <div>
      <h2>UseRef</h2>

      <input type="text" ref={inputref} placeholder="Enter your name.." />

      <button onClick={focusInput}>Click</button>
    </div>
  );
```

```
};

export default Usererefff;
```

📘 Theory

1. useReducer

- **Definition**: Alternative to useState for complex state logic.
- **Syntax**:

js

const [state, dispatch] = useReducer(reducer, initialState);

- **Reducer function**: (state, action) => newState.
- **Dispatch**: Sends actions with type and optional payload.
- Best for: multiple state transitions, complex updates, or when state depends on previous state.

2. useRef

- **Definition**: Returns a mutable ref object that persists across renders.
- **Common uses**:
    - Accessing DOM elements (inputRef.current.focus()).
    - Storing mutable values without causing re-renders.
- Unlike state, updating .current does not trigger re-render.

3. useEffect

- **Definition**: Side-effect hook for data fetching, subscriptions, or DOM updates.
- **Syntax**:

js

useEffect(() => {

  // side effect

}, [dependencies]);

- **Dependencies array**: Controls when effect runs.
- Empty array [] → runs once on mount.

- No array → runs on every render.

Coding Round Questions

- useReducer → When should you use useReducer instead of useState?
- useRef → What are common use cases for useRef?
- useEffect → How does the dependency array affect useEffect execution?
- Hooks comparison → Compare useState, useReducer, and useRef.
- Data fetching → How do you fetch data using useEffect?
- Performance → How can hooks improve performance in React?

⚡ **Machine Coding Round Challenge**

Problem: Todo App with Hooks

Requirements:

1. Use useReducer to manage todo list state (add, delete, toggle).
2. Use useRef to focus input after adding a todo.
3. Use useEffect to fetch initial todos from an API.
4. Display todos with toggle functionality.